

1. Overview

Scenario.

You are building a backend for an application used by a Romanian insurance company that manages **building insurance policies**.

There are two main user groups:

1. **Brokers**
 - Manage clients.
 - Manage buildings (objects of insurance).
 - Create and manage policies.
 2. **Administrators**
 - Maintain metadata (currencies, fees/coefficients, risk factors).
 - Manage brokers (activate/deactivate).
 - View reports (e.g., number and total value of policies per country/county/city, per broker, per period).
-

2. Learning goals

By implementing this exercise, the learner should practice:

1. Designing clear **domain models** and relationships.
 2. Exposing **REST API endpoints** for different user roles.
 3. Implement according API first - design approach
 4. Handling **basic business rules** around policies and premiums.
 5. Writing code that is **easy to extend**: separation between domain, data access, and Web API.
 6. Implementing **unit tests** for core logic and **integration tests** for API endpoints and persistence. At least 80% code coverage is required (this percentage can grow depending on the technical stack used)
 7. Thinking about **code quality**, **readability**, and **maintainability** from the start.
-

3. Domain overview & core concepts

The system manages:

- **Clients** – people or companies that own buildings and buy policies.

- **Buildings** – the insured objects.
 - **Policies** – contracts covering buildings for a period, with a premium and currency.
 - **Geography** – country, county, city.
 - **Brokers** – intermediaries that sell policies.
 - **Administrators** – internal staff who manage configuration and reporting.
 - **Metadata** – currencies, fees/percentages, risk coefficients, etc.
-

4. Functional requirements – Admin-facing endpoints

4.1 Reporting

1. Administrators must be able to **get aggregated reports** for policies:

At minimum:

- Total number of policies and total final premium **per country** in a date range (policy start date).
- Total number of policies and total final premium **per county** in a date range.
- Total number of policies and total final premium **per city** in a date range.
- Total number of policies and total final premium **per broker** in a date range.

All the reports will have **currency** as a second **grouping key**.

2. For each report, administrators must be able to **filter by**:
 - Policy status (Active, Expired, Cancelled).
 - Currency.
 - Optional: building type.
 3. Reports should return **simple tabular data**, suitable for further export (e.g., list of rows with grouping fields, counts, and totals).
-

5. Non-functional & code-quality requirements

The code should aim for:

1. **Separation of concerns**
 - Keep business logic out of controllers.
 - Use services or application layer classes to handle reporting logic.
2. **Maintainability**
 - Clear naming.

- Small methods with single responsibility.
 - Avoid duplication
 - 3. **Scalability (at a basic level)**
 - Avoid loading too much data into memory unnecessarily.
 - 4. **Extensibility**
 - New filters should require **minimal changes**.
 - Use design patterns (e.g. Strategy) to structure your code to adapt to new features.
 - 5. **Validation**
 - Validate incoming data:
 - Required or optional query params (status, building type).
 - Reasonable ranges (from <= to).
 - Return clear error messages.
 - 6. **Error handling and logging**
 - Centralised error handling (e.g., a middleware or equivalent).
 - Log key events
 - 7. **Security (basic level)**
 - Assume that some form of authentication will be used.
 - Endpoints for brokers and admins should be **clearly separated**, ready to apply authorization rules.
-

6. API specification (high-level)

Examples (developers can choose exact routes):

8.6 Admin – Reports

- GET /api/admin/reports/policies-by-country?from=&to=&status=¤cy=&buildingType=
- GET /api/admin/reports/policies-by-county?from=&to=&status=¤cy=&buildingType=
- GET /api/admin/reports/policies-by-city?from=&to=&status=¤cy=&buildingType=
- GET /api/admin/reports/policies-by-broker?from=&to=&status=¤cy=&buildingType=

Each report endpoint returns a list with:

- Grouping keys (country/county/city/broker) and Currency.
 - PolicyCount.
 - TotalFinalPremium.
 - TotalFinalPremiumInBaseCurrency
-

Responses should include clear status codes:

- 200 OK (success).
 - 201 Created (for creations).
 - 400 Bad Request (validation errors).
 - 404 Not Found.
 - 500 for unexpected server errors.
-

7. Testing plan (separate: unit & integration)

7.1 Unit tests

Goal: Verify **business logic** in isolation.

Focus on:

1. **Reporting logic (aggregations)**
 - Given a small set of policy data in memory, verify:
 - Grouping and sums per geography or broker are correct.
 - Filters by status and date range are applied.

What to mock for unit tests:

- Data access abstractions (clients, policies, fees, risk factors, geography).
 - Logging interfaces.
-

7.2 Integration tests

Goal: Verify the Web API endpoints work correctly **end-to-end**: routing, model binding, validation, and database persistence.

Suggested integration tests:

1. **Reporting endpoints**
 - Seed several policies with different cities and brokers.
 - Call `policies-by-city` and `policies-by-broker`.
 - Assert:
 - Aggregated counts and totals match the seeded data.
 - Filters (date range, currency, status) change results as expected.

Integration test considerations:

- Use an **in-memory or test-specific database**.
 - Run setup before tests.
 - Clean database state between tests.
-

8. Acceptance criteria (checklist)

For the exercise to be considered complete:

1. **Reports**
 - Admin endpoints provide aggregated data (policy count and total premium) by geography and broker with filters.
 2. **Quality**
 - Code is structured in clear layers (domain, application/services, infrastructure, Web).
 - Validation is implemented and tested.
 - Centralised error handling present.
 3. **Tests**
 - Unit tests cover reporting logic.
 - Integration tests cover main flows and reports.
 - Tests are easy to run from the solution.
-
-